

10 Snowflake SQL Tricks That Instantly Improve Query Performance

If your Snowflake queries are becoming slower as data grows, you are not alone.

Many developers focus only on warehouse size when optimizing performance. But in reality, small SQL improvements can reduce execution time dramatically while also lowering compute costs.

In this article, we'll explore 10 practical Snowflake SQL tricks that instantly improve query performance — with examples you can start using today.

1. Select Only Required Columns

One of the most common performance mistakes is using:

```
SELECT *  
FROM SALES_DATA;
```

This forces Snowflake to scan unnecessary columns.

Instead:

```
SELECT CUSTOMER_ID, ORDER_DATE, TOTAL_AMOUNT  
FROM SALES_DATA;
```

Why It Helps

Snowflake uses columnar storage.

Reading fewer columns means:

- Less data scanned
- Faster execution
- Lower cost

This becomes extremely important for wide tables containing hundreds of columns.

2. Filter Early Using WHERE Clause

Bad approach:

```
SELECT *  
FROM ORDERS  
QUALIFY ROW_NUMBER() OVER (
```

```

    PARTITION BY CUSTOMER_ID
    ORDER BY ORDER_DATE DESC
) = 1
AND ORDER_DATE >= '2025-01-01';

```

Better approach:

```

SELECT *
FROM ORDERS
WHERE ORDER_DATE >= '2025-01-01'
QUALIFY ROW_NUMBER() OVER (
    PARTITION BY CUSTOMER_ID
    ORDER BY ORDER_DATE DESC
) = 1;

```

Why It Helps

Filtering earlier reduces:

- Rows processed
- Memory usage
- Window function workload

Always reduce dataset size before expensive operations.

3. Avoid Using Functions on Filter Columns

Slow query:

```

SELECT *
FROM CUSTOMERS
WHERE YEAR(CREATED_AT) = 2025;

```

Optimized query:

```

SELECT *
FROM CUSTOMERS
WHERE CREATED_AT >= '2025-01-01'
AND CREATED_AT < '2026-01-01';

```

Why It Helps

Functions on columns prevent:

- Micro-partition pruning
- Efficient scanning

Snowflake performs best when predicates directly reference raw column values.

4. Use QUALIFY Instead of Nested Subqueries

Traditional approach:

```
SELECT *
FROM (
    SELECT *,
           ROW_NUMBER() OVER (
               PARTITION BY CUSTOMER_ID
               ORDER BY ORDER_DATE DESC
           ) AS RN
    FROM ORDERS
)
WHERE RN = 1;
```

Better Snowflake-native approach:

```
SELECT *
FROM ORDERS
QUALIFY ROW_NUMBER() OVER (
    PARTITION BY CUSTOMER_ID
    ORDER BY ORDER_DATE DESC
) = 1;
```

Why It Helps

QUALIFY:

- Simplifies query structure
- Improves readability
- Often reduces unnecessary intermediate processing

It's one of Snowflake's most useful SQL features.

5. Use APPROX Functions for Large Analytics

Exact distinct counts are expensive:

```
SELECT COUNT(DISTINCT USER_ID)
FROM EVENT_LOGS;
```

Faster alternative:

```
SELECT APPROX_COUNT_DISTINCT(USER_ID)
FROM EVENT_LOGS;
```

Why It Helps

Approximation algorithms:

- Consume less memory
- Execute faster
- Scale better on huge datasets

Perfect for dashboards and analytics where tiny inaccuracies are acceptable.

6. Reduce Data Before JOINS

Inefficient approach:

```
SELECT *
FROM ORDERS O
JOIN CUSTOMERS C
ON O.CUSTOMER_ID = C.CUSTOMER_ID
WHERE O.ORDER_DATE >= '2025-01-01';
```

Better approach:

```
WITH FILTERED_ORDERS AS (
    SELECT *
    FROM ORDERS
    WHERE ORDER_DATE >= '2025-01-01'
)

SELECT *
FROM FILTERED_ORDERS O
JOIN CUSTOMERS C
ON O.CUSTOMER_ID = C.CUSTOMER_ID;
```

Why It Helps

Smaller joins mean:

- Less shuffling
- Lower memory usage
- Faster execution

Always reduce rows before joins whenever possible.

7. Prefer UNION ALL Over UNION

Expensive query:

```
SELECT * FROM JAN_SALES
UNION
SELECT * FROM FEB_SALES;
```

Optimized query:

```
SELECT * FROM JAN_SALES
UNION ALL
SELECT * FROM FEB_SALES;
```

Why It Helps

UNION performs duplicate elimination, which requires:

- Sorting
- Additional memory
- More compute

If duplicates are acceptable, always use UNION ALL.

8. Cluster Large Tables Strategically

For very large tables, clustering can dramatically improve pruning.

Example:

```
ALTER TABLE SALES
CLUSTER BY (ORDER_DATE);
```

Why It Helps

Clustering improves:

- Micro-partition pruning
- Query scan efficiency
- Performance on filtered queries

Best for tables:

- With billions of rows
- Frequently filtered on the same columns

Avoid excessive clustering on small tables.

9. Use RESULT CACHE Whenever Possible

Snowflake automatically caches query results.

Running the exact same query again:

```
SELECT *  
FROM DAILY_METRICS  
WHERE METRIC_DATE = CURRENT_DATE;
```

may return instantly from cache.

Tips to Maximize Cache Usage

- Avoid unnecessary query changes
- Use consistent formatting
- Reuse stable SQL patterns

Cached results can return in milliseconds without consuming compute.

10. Analyze Query Profile Regularly

The Query Profile is one of the most powerful optimization tools in Snowflake.

It helps identify:

- Expensive joins
- Large scans
- Data skew
- Spilling to disk
- Bottlenecks

What to Watch

Focus on:

- Partitions scanned
- Bytes spilled
- Join explosion
- Long-running operators

Even a well-written query can hide expensive execution steps.

Bonus Tips

Use Search Optimization Service Carefully

Useful for highly selective lookups:

```
ALTER TABLE CUSTOMERS  
ADD SEARCH OPTIMIZATION;
```

Best for:

- Point lookups
- Text searches
- Highly selective filters

But avoid enabling it everywhere because it increases storage costs.

Common Performance Mistakes

Mistake	Better Approach
SELECT *	Select required columns
UNION everywhere	Use UNION ALL
Functions in WHERE clause	Use direct predicates
Large joins without filtering	Filter first
Exact analytics on huge data	Use APPROX functions

Final Thoughts

Snowflake performance optimization is often about reducing unnecessary work.

The biggest improvements usually come from:

- Scanning less data
- Filtering earlier
- Simplifying execution plans
- Using Snowflake-native features properly

You do not always need larger warehouses.

Sometimes a simple SQL rewrite can make queries run 10x faster while significantly reducing cost.

Which Optimization Gives the Biggest Impact?

In real-world Snowflake workloads, these usually provide the biggest gains:

1. Eliminating **SELECT ***
2. Better filtering for partition pruning
3. Reducing JOIN size
4. Using clustering strategically
5. Leveraging caching effectively

Start with these first before scaling warehouse size.